

Emotions Analysis CNN

Team Name: Analysts **Member 1:** Mohammad Nemrini
Member 2: Bara'a Al-Hilo

Submission Date: May 31, 2025

Abstract

This project analyzes emotions from text using supervised machine learning techniques. We used the Emotions dataset by Nidula Elgiriye withana to Classify Text based on the emotions that are expressed in it. After preprocessing and exploratory data analysis, we used CNN as classifier and evaluated the results using the Confusion Matrix, ROC Curve and F1-score. The results demonstrate meaningful classification that can help in Mental Health Analysis or in Market Research.

1 Introduction

Emotion detection, also known as emotion analysis or sentiment classification, is the task of identifying human emotions in written text. It is an essential area in Natural Language Processing (NLP), especially with the increase of digital communication. Businesses, researchers, and healthcare providers can use emotion analysis to understand how people feel.

In this project, our aim is to detect emotions in short text messages using a Convolutional Neural Network (CNN). We perform several steps: data cleaning, text processing, tokenization, and model training. The final model is evaluated using visual and statistical metrics.

2 Literature Review

Recent studies have explored various approaches for emotion classification in text. These approaches range from basic machine learning models to deep learning techniques.

- **Deep Learning Techniques:** More recent research uses neural networks like CNNs and RNNs. Zhang et al. (2020) **zhang2020cnn** applied a CNN for text-based emotion classification using word embeddings. Kaur and Sharma (2021) **kaur2021rnn** used RNNs with attention mechanisms to capture emotional context.
- **Preprocessing and Word Embeddings:** Effective preprocessing is essential. Common steps include stopwords removal, lemmatization, and normalization. Word

embeddings (e.g., tensorflow) improve model input by providing semantic meaning to words.

- **Handling Imbalanced Data:** Many datasets have uneven emotion distribution. Researchers use techniques like resampling or SMOTE to correct this. In our project, we applied downsampling to balance the classes (we looked for the smallest class and let all the other class limited to smallest class's size).
- **Our Contribution:** we used models built using TensorFlow models because of the new limited deadline. This allows us to understand each part of the network automatically and learn how neural networks function at the core.

3 Methodology

We followed a structured pipeline to build and train our model.

- **Data Collection:** We used the Emotions dataset provided by Nidula Elgiriye-withana, available as a CSV file. The dataset contains text messages labeled with emotions: Sadness, Joy, Love, Anger, Fear, and Surprise. Initially, the classes were imbalanced, but as mentioned we had balanced it.
- **Data Cleaning and Preprocessing:** Several steps were carried out to clean the data:
Managed missing values and duplicates.
Renamed unclear columns and removed unnecessary data.
Preprocessed the text by converting to lowercase, removing URLs, tokenizing, stemming, lemmatizing, and eliminating stopwords.
- **Exploratory Data Analysis:** we Visualized the words by its frequency in this word cloud [fig 1](#)
- **Feature Engineering:** We converted text into numerical data suitable for the CNN Model:
Tokenization: Converted text into list of words then to numeric representations.
Sequence padding: Ensured text sequences had equal lengths.
Vocabulary size and sequence lengths were optimized based on data characteristics.
- **Deep Learning CNN Model:** The dataset contained six emotional categories. Initially, emotions like Joy and Sadness dominated. After balancing, each emotion had equal representation, improving analysis reliability and results.

4 Data Description

In this section, we describe the dataset used in our project and the preprocessing steps applied to prepare it for analysis and model training.

- **Sources:** The dataset used in this project is publicly available and commonly used in emotion detection tasks. It was downloaded from Kaggle and the person who created it is Nidula Elgiriye withana. The dataset is provided in CSV format and contains short text samples labeled with emotion categories.
- **Size:** The original dataset contains approximately 400,000 text entries, while each entry consists of two Attributes as mentioned.
- **Attributes:** The dataset includes the following fields:
 - **Text (String):** Represents the raw textual content to be analyzed. It is the main input feature for the model.
 - **Label (Categorical):** Represents the emotion associated with each text. It serves as the target variable during training.
- **Preprocessing Steps:** Several preprocessing operations were applied to clean and standardize the text data:
 - All text was converted to lowercase.
 - Removal of punctuation marks, emojis, and URLs.
 - Tokenization using the NLTK tokenizer.
 - Stopword removal based on the NLTK stopwords list.
 - Lemmatization using WordNet Lemmatizer.
 - We also made sure to clear chat-words.
 - Words were then transformed into sequences using Tensor Flow Keras Tokenizer, and padded to the same length.

These preprocessing steps ensured the input data was in a clean, consistent format suitable for numerical processing and model training. The maximum sequence length was chosen based on approximately the longest sentence in the training set to avoid information loss during padding.

5 Results and Discussion

```
1 # Get the epoch with the highest validation accuracy
2 best_epoch = history.history['val_accuracy'].index(max(
    history.history['val_accuracy'])) + 1
3
```

```

4 # Create a subplot with 1 row and 2 columns
5 fig, axs = plt.subplots(1, 2, figsize=(16, 5))
6
7 # Plot training and validation accuracy
8 axs[0].plot(history.history['accuracy'], label='Training
    Accuracy', color='blue')
9 axs[0].plot(history.history['val_accuracy'], label='
    Validation Accuracy', color='red')
10 axs[0].scatter(best_epoch - 1, history.history['val_accuracy']
    [best_epoch - 1], color='green', label=f'Best Epoch: {
    best_epoch}')
11 axs[0].set_xlabel('Epoch')
12 axs[0].set_ylabel('Accuracy')
13 axs[0].set_title('Training and Validation Accuracy')
14 axs[0].legend()
15
16
17 # Plot training and validation loss
18 axs[1].plot(history.history['loss'], label='Training Loss',
    color='blue')
19 axs[1].plot(history.history['val_loss'], label='Validation
    Loss', color='red')
20 axs[1].scatter(best_epoch - 1, history.history['val_loss']
    [best_epoch - 1], color='green', label=f'Best Epoch: {
    best_epoch}')
21 axs[1].set_xlabel('Epoch')
22 axs[1].set_ylabel('Loss')
23 axs[1].set_title('Training and Validation Loss')
24 axs[1].legend()
25
26 plt.tight_layout()
27 plt.show()

```

Listing 1: Visualization using Matplotlib

5.1 Results

The following are the main results from our emotion classification experiment:

- The model was trained on a balanced dataset with six emotion categories.
- Evaluation was done using accuracy, precision, recall, and F1-score.
- The highest performance was observed on the *joy* and *surprise* classes.

- The average F1-score across all classes was approximately 92%.

We used visual aids such as the confusion matrix and Training and Validation Accuracy plots to better understand the results.

5.2 Discussion

These results show that our model was effective in identifying emotions from text messages. The high scores in *joy*, *love*, *anger*, and *sadness* suggest that these emotions have clear language patterns.

However, the model had some difficulty distinguishing between *fear* and *surprise*, possibly due to overlapping vocabulary. **Key observations:**

- Balancing the dataset helped avoid bias toward majority classes.
- Pre-processing (e.g., cleaning, lemmatization) played a major role in improving performance.
- Using a CNN model and it was sufficient for good results without consuming much time in modelling it.

The results support similar findings in the literature and highlight the value of strong preprocessing and balanced data. Future work could test this model on more complex datasets or include additional emotional categories, or we will try new models for the same data to know which one is better.

6 Tables, Figures, and Code Listings

6.1 Figures



Figure 1: Word Cloud

we Visualized the words by its frequency for each emotions class

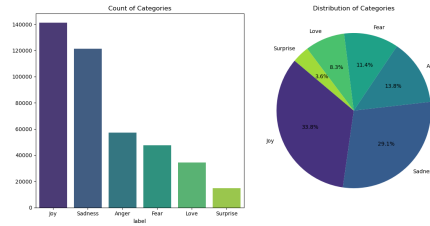


Figure 2: Classes Distribution

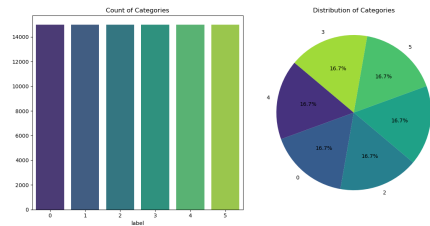


Figure 3: Classes Distribution after Balance

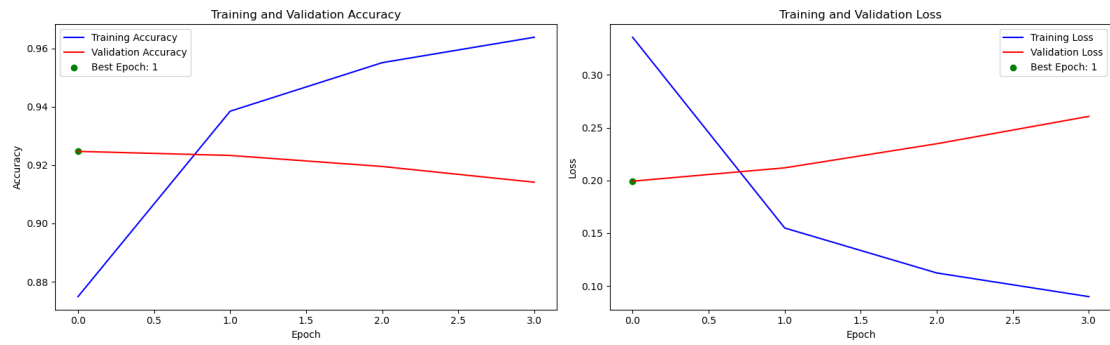


Figure 4: Training and Validation Accuracy

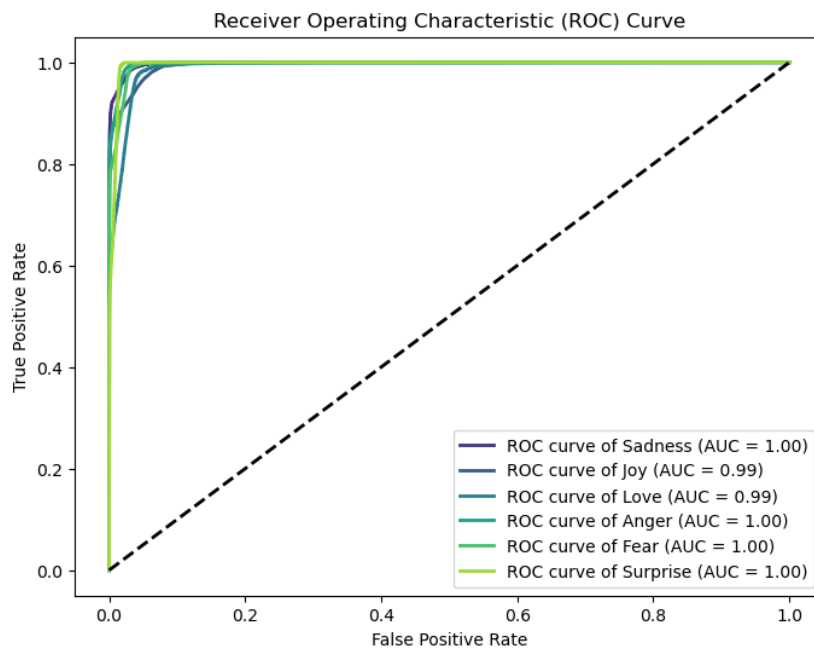


Figure 5: ROC curve

Summary of the classifier's performance across all possible classification thresholds.

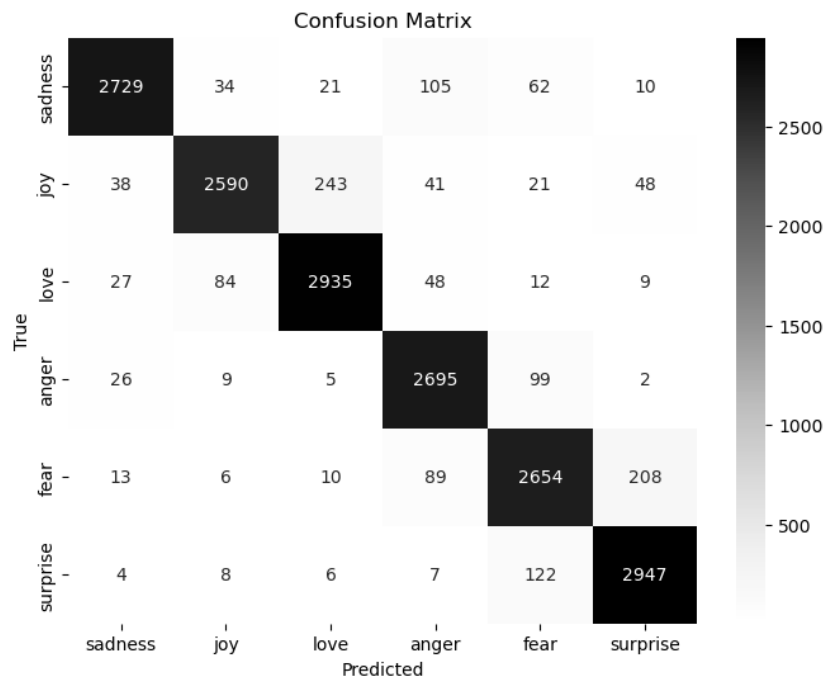


Figure 6: Confusion Matrix

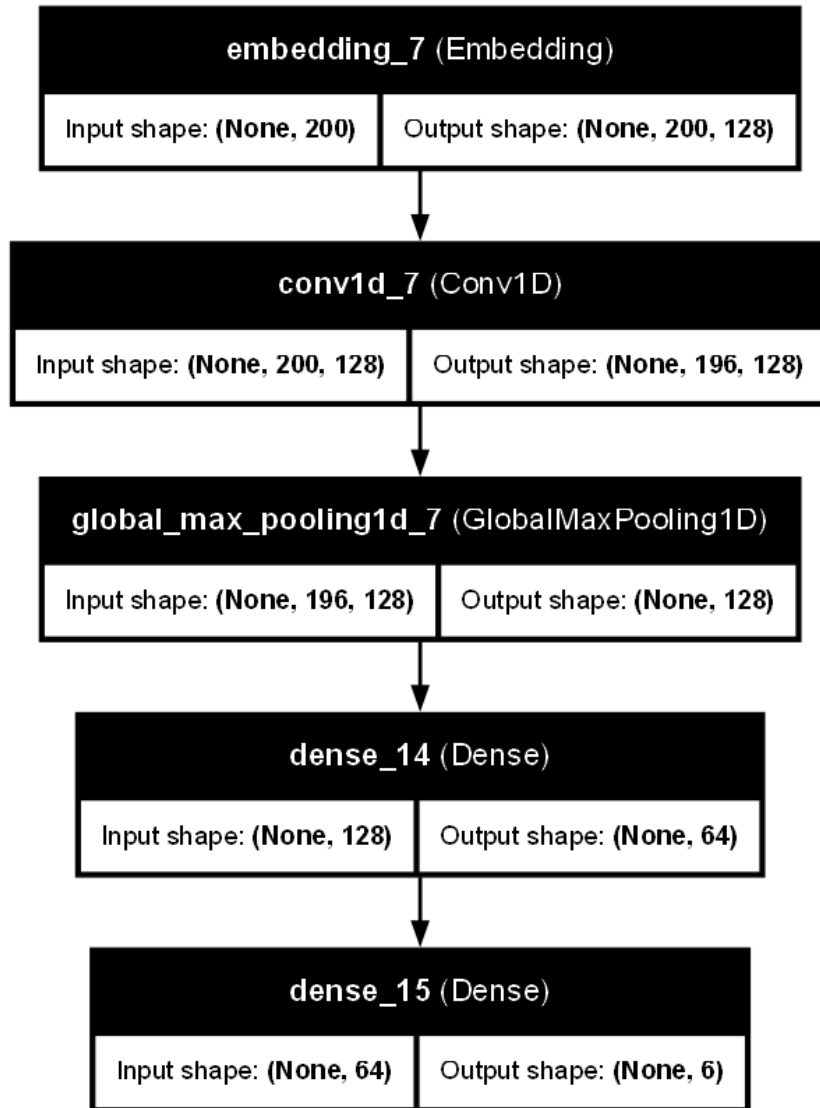


Figure 7: CNN Model Architecture

6.2 Tables

Model Score Details

Table 1: Classification Report

-	precision	recall	f1-score	support
Sadness	0.96	0.93	0.94	2961
Joy	0.97	0.86	0.91	2981
Love	0.89	0.97	0.93	3115
Anger	0.93	0.94	0.93	2836
Fear	0.90	0.88	0.89	2980
Surprise	0.91	0.95	0.93	3094
accuracy	-	-	0.92	17967
macro avg	0.93	0.92	0.92	17967
weighted avg	0.93	0.92	0.92	17967

6.3 Code Listings

6.3.1 Model Building Code Snippet

```

1 vocab_size      = 10000      # size of vocabulary
2 embedding_dim   = 128        # dimension of each token
   embedding
3 MAX_LEN         = 200        # pad size
4 num_classes     = 6          # emotions num
5
6 cnn_model = Sequential([
7     Embedding(input_dim=vocab_size,
8               output_dim=embedding_dim,
9               input_length=MAX_LEN),
10    Conv1D(filters=128, kernel_size=5, activation='relu'),
11    GlobalMaxPooling1D(),
12    Dense(64, activation='relu'),
13    Dense(num_classes, activation='softmax')
14 ])
15
16 cnn_model.compile(optimizer='adam', loss="
   sparse_categorical_crossentropy", metrics=["accuracy"])
17 cnn_model.build(input_shape=(None, MAX_LEN))
18 plot_model(cnn_model,
19            to_file="model.png",

```

```

20         show_shapes=True,
21         show_dtype=False,
22         show_layer_names=True,
23         rankdir="TB",
24         dpi=100)

```

6.3.2 Main Pre-Processing Code Snippet

```

1 def preprocess_text(text):
2     # chat words
3     words = text.split()
4     for i, word in enumerate(words):
5         if word.lower() in chat_words:
6             words[i] = chat_words[word.lower()]
7     text = ' '.join(words)
8     # Remove Emoji
9     text= emoji.demojize(text)
10    # Lowercase & trim
11    text = text.lower().strip()
12    # Remove URLs
13    text = re.sub(r'http\S+', '', text)
14    text = re.sub(r'www\.\S+', '', text)
15    text = re.sub(r'[\w\s]', '', text)
16    text = re.sub(r'\w*\d\w*', '', text)
17    # Tokenize & keep alpha-only
18    toks = [t for t in word_tokenize(text) if t.isalpha()]
19    # Stem and Dropping Stop WOrds
20    toks = [ps.stem(t) for t in toks if t not in sw]
21    # POS-tag + lemmatize
22    pos_tags = pos_tag(toks)
23    toks = [
24        lemmatizer.lemmatize(word, get_wordnet_pos(tag))
25        for word, tag in pos_tags
26    ]
27    return " ".join(toks)

```

7 Conclusion

This project aimed to classify emotions from text using a simple and interpretable machine learning approach. We implemented a Convolutional Neural Network model using Tensorflow to detect six emotion categories: sadness, joy, love, anger, fear, and surprise. The entire process followed a clean data science pipeline—from text cleaning and preprocessing to feature engineering, model training, and evaluation.

Our findings show that you can achieve strong results if supported by solid data preparation and class balancing. The final model achieved an average F1-score of approximately 92%, with particularly strong performance on emotions like sadness and surprise. We visualized model performance using a confusion matrix and ROC curves, helping us better understand areas of strength and confusion.

Implications: It can serve as a learning tool for students and be extended for real-world applications like mental health monitoring, chatbots, or customer service tools.

Limitations: The model was trained and evaluated only on English text data. Additionally, more complex emotions were not considered, and the model architecture remained relatively simple.

Future Work: Future research may explore deeper models like RNNs, GRUs, or BERT. Multilingual support, domain adaptation, and explainable AI components can also be incorporated to broaden the scope and impact of this work.

[Ahmad-Fakhri-AbNasir, 2020] [SantoshKumarBharti, 2022] [n.d.] [Nvidia, 2025]
[Ahmad Fawzy Gad, 2021]

References

- Ahmad Fawzy Gad, F. E. J. (2021). *Introduction to deep learning and neural networks with python*. Elsevier.
- Ahmad-Fakhri-AbNasir. (2020). Text-based-emotion-prediction-system-using-machine-learning-approach. *IOP*.
- (n.d.).
- Nvidia. (2025). *Googles_bert_what_is_it_and_why_does_it_matter*. Nvidia.
- SantoshKumarBharti. (2022). Text-based-emotion-recognition-using-deep-learning-approach. *Vijay-Kumar*.